

Atividade 1 (AV1) — Relatório Técnico

Recuperação de contexto para IA generativa em materiais educacionais abertos

Corretude, eficiência e recuperação de contexto:
um estudo comparativo de estratégias de ordenação e busca sobre um livro-texto aberto

João Cosme Sena Sá · Eduardo Henrique do Lago Silva · Helena Carvalho Leal
Hernandison da Silva Bispo · Nadianne Maria dos Santos Galvão · Rafael Takeguma Goto

Tema	8 — Materiais educacionais abertos
Corpus	<i>Pense Python</i> (tradução livre da 3ª ed. de <i>Think Python</i>)
Repositório	github.com/joaocss/PAA_UFS_2026_2_SenaSa_Joao_Bispo_Hernandison_Galvao_Nadianne_Goto_Rafael_Leal_Helena_Silva_Eduardo
Licença do código	MIT · Versão/tag: v0.1.0
Vídeo da atividade	URL a preencher até 22/09/2026 (também em README, VIDEO.md e Google Classroom)

Estado desta versão do relatório (checkpoint de 04/09/2026). Este documento consolida a **especificação formal**, os **algoritmos e pseudocódigos**, a **prova de corretude**, a **análise no modelo RAM**, as **recorrências**, a **metodologia experimental** e a **relação com RAG** — partes que não dependem da execução do pipeline. As figuras 1 e 2 são **curvas e diagramas teóricos**, derivados das funções de complexidade, e não medições. As tabelas da Seção 12 (resultados empíricos) trazem a estrutura definitiva de colunas e estão marcadas *"a preencher após a execução"*; os números serão inseridos após rodar `scripts/reproduzir_tudo.py`, conforme o cronograma da disciplina (demonstração inicial no checkpoint de 10/09/2026 e resultados completos até 23/09/2026). Nenhum dado experimental foi estimado ou inventado.

Sumário

1. Identificação da equipe
2. Tema, corpus e fonte
3. Repositório, licença, versão e data de acesso
4. Definição formal do problema
5. Representação dos dados e estratégia de chunking
6. Algoritmos e pseudocódigo
7. Justificativa de corretude
8. Análise no modelo RAM
9. Melhor, pior e caso médio
10. Recorrências
11. Metodologia experimental
12. Resultados e gráficos
13. Discussão de escalabilidade
14. Relação com IA generativa e RAG
15. Limitações e ameaças à validade
16. Declaração de uso de IA generativa
17. Contribuição individual
18. Vídeo da atividade
19. Referências

1. Identificação da equipe

Equipe de seis discentes do curso de Ciência da Computação (PROCC/UFS), matriculados na disciplina Projeto e Análise de Algoritmos no período 2026.2. A modalidade da atividade é trabalho em equipe (5 a 7 integrantes).

Integrante	Frete principal
João Cosme Sena Sá	Coordenação e entrega; dados de teste e pacote; índice invertido e busca binária no vocabulário (Config. 2)
Eduardo Henrique do Lago Silva	Experimentos, tabelas, gráficos e baseline de referência scikit-learn (Config. 4)
Helena Carvalho Leal	Consultas, julgamentos de relevância (qrels), métricas (Precision@k), relação com IA e limitações
Hernandison da Silva Bispo	Ingestão, normalização, fragmentação e manutenção do relatório
Nadianne Maria dos Santos Galvão	Corretude, modelo RAM, complexidade e recorrências
Rafael Takeguma Goto	Busca linear, Insertion Sort, Merge Sort, pontuação e consulta (Config. 1 e 3)

A divisão acima é uma **proposta de frentes** para organizar o trabalho; a revisão de código, da prova e dos resultados é responsabilidade coletiva. A tabela de contribuição individual **com evidências** (commits, arquivos, execuções) é mantida em `docs/CONTRIBUICOES.md` e resumida na Seção 17.

2. Tema, corpus e fonte

O tema escolhido é o de número 8 do enunciado, **materiais educacionais abertos**. O corpus é o livro *Pense Python*, tradução livre da 3ª edição de *Think Python*, de Allen B. Downey, traduzida por Rodrigo Castelan Carlson. O material reúne conteúdo didático público de introdução à programação em Python, sem qualquer dado sensível, pessoal ou sigiloso.

Campo	Registro
Nome	<i>Pense Python</i> — tradução livre da 3ª edição de <i>Think Python</i>
Autor / Tradução	Allen B. Downey / Rodrigo Castelan Carlson
Fonte	rodrigocarlsen.github.io/PensePython3ed · github.com/rodrigocarlsen/PensePython3ed
Commit de referência	<code>cfde2c450bf4f2ea65326da88e9968f400597e92</code> (branch <code>main</code>)
Recorte	21 notebooks Jupyter: prefácio, introdução e capítulos 1 a 19 (<code>capitulos/*.ipynb</code>)
Tamanho auditado	817.578 bytes · 2.789 células · ~73.867 palavras (texto e código)
SHA-256 do recorte	<code>2010626258...5768dc5</code> (agregado; ver <code>data/corpus_manifest.csv</code>)
Idioma	Português brasileiro (com estrangeirismos técnicos do Python/Jupyter)
Licença do texto / do código	CC BY-NC-SA 4.0 / MIT
Data de acesso	02/09/2026 (aquisição registrada); planejamento em 01/09/2026
Dados sensíveis	Nenhum — conteúdo didático público

Corpus não versionado, mas reproduzível. Como a licença do texto é não comercial e com compartilhamento pelas mesmas condições (CC BY-NC-SA), o texto não é redistribuído no repositório da equipe (cujo código é MIT). Em vez disso, o script `scripts/baixar_corpus.py` baixa o material no commit fixado e o arquivo `data/corpus_manifest.csv` guarda o SHA-256 de cada notebook e um **hash agregado do recorte**, permitindo conferir que qualquer download reproduz exatamente o mesmo conjunto auditado. Trechos citados no relatório preservam atribuição ao autor e ao tradutor.

Pergunta(s) de recuperação. As consultas seguem o formato “tira-dúvidas” de um estudante sobre o livro, por exemplo: “*como funciona um laço for em Python?*”, “*o que é uma função recursiva?*”, “*como tratar exceções com try e except?*”. O conjunto completo de 30 consultas está em `data/queries.csv`, redigido **antes** de qualquer resultado ser observado.

3. Repositório, licença, versão e data de acesso

Item	Registro
URL do repositório	https://github.com/joaocss/PAA_UFS_2026_2_SenaSa_Joao_Bispo_Hernandison_Galvao_Nadianne_Goto_Rafael_Leal_Helena_Silva_Eduardo
Licença do código	MIT (arquivo <code>LICENSE</code>)
Versão / tag	<code>v0.1.0</code> (<code>CITATION.cff</code> , <code>date-released: 2026-09-03</code>)

Commit/tag/release do relatório	<i>a fixar na entrega final (tag da release de 23/09/2026)</i>
Data de acesso ao corpus	02/09/2026 (a reconfirmar no dia da aquisição)
Ambiente travado	requirements.lock : pytest 8.3.4, scikit-learn 1.6.1, matplotlib 3.10.0, numpy 2.2.2

O repositório não contém chaves, senhas, tokens, dados pessoais ou artefatos sem licença compatível. Arquivos grandes (o corpus baixado e dados intermediários) são controlados por `.gitignore`; as instruções de obtenção e reprodução estão no `README.md`.

4. Definição formal do problema

O protótipo é um **recuperador de contexto**: dada uma consulta, devolve os trechos mais relevantes do corpus, ordenados por relevância. Não é um chatbot; o interesse está nas operações algorítmicas anteriores à geração de texto.

4.1 Entradas e saída

- **Corpus** $C = \langle c_1, \dots, c_N \rangle$: sequência de N chunks. Cada chunk c_i tem um `chunk_id` textual único, um `source_order` id_i inteiro único em $0..N-1$ (usado no desempate), o texto normalizado e $origem_i = (\text{notebook}, \text{seção}, \text{offset})$.
- **Consulta** q : cadeia de caracteres em português; após normalização vira um multiconjunto de m termos.
- $k \in \mathbb{N}$: número máximo de trechos a devolver.
- **Saída** R : sequência de até k chunks de C , sem repetição, ordenada de forma não crescente pela função de relevância.

4.2 Função de relevância

A pontuação é **TF-IDF** lexical, definida pela equipe e usada de forma idêntica nas três configurações obrigatórias:

$$tf(t, c) = 1 + \ln f_{t,c} \text{ (se } f_{t,c} > 0; \text{ senão } 0) \quad \cdot \quad idf(t) = \ln \left(\frac{(N+1)}{(df(t)+1)} \right) + 1$$

$$score(q, c) = \sum_{t \in q} tf(t, c) \cdot idf(t)$$

em que $f_{t,c}$ é a frequência do termo t no chunk c , e $df(t)$ é o número de chunks que contêm t . Termos da consulta ausentes do vocabulário não contribuem (contribuição 0). As formas suavizadas ($\ln f$ no tf e $\ln((N+1)/(df+1))+1$ no idf) são as mesmas do TF-IDF do scikit-learn, o que mantém limpa a comparação com a Config. 4 e evita zerar um termo presente em todos os chunks. A **ordem total** $\preceq$ usada na ordenação é:

$$c_a \preceq c_b \iff score(q, c_a) > score(q, c_b) \vee (score \text{ iguais} \wedge id_a < id_b)$$

onde id denota o `source_order` do chunk. O desempate determinístico por `source_order` crescente garante que o resultado independe da ordem de entrada e da configuração usada.

4.3 Pré-condições, pós-condições e casos de borda

Categoria	Especificação
Pré-condições	$N \geq 0$; $k \geq 0$; todo $score$ é um número real finito; os <code>source_order</code> são únicos; o comparador $\preceq$ é uma ordem total sobre os chunks.
Pós-condições	Seja n' o número de chunks com $score > 0$. Então $ R = \min(k, n')$; todo elemento de R pertence a C e é único; R está ordenado por $\preceq$; e não existe chunk fora de R com relevância maior (por $\preceq$) que qualquer elemento de R . Ou seja, R são exatamente os $\min(k, n')$ melhores.
Casos de borda	consulta vazia após normalização $\rightarrow R = \{\}$; $k = 0 \rightarrow R = \{\}$; $k > n' \rightarrow$ devolve os n' com $score > 0$; todos os $score$ 0 (nenhum termo casa) $\rightarrow R = \{\}$; empates massivos $\rightarrow$ resolvidos por <code>source_order</code> ; termo fora do vocabulário $\rightarrow$ ignorado.
Hipótese sobre os dados	Para a vantagem do índice invertido (Config. 2) supõe-se esparsidade : os termos de consulta típicos ocorrem em poucos chunks ($df \ll N$), de modo que o conjunto de candidatos é bem menor que N . A correteza não depende dessa hipótese; apenas a eficiência.

5. Representação dos dados e estratégia de chunking

5.1 Normalização

A normalização é deliberadamente simples, para poder ser auditada:

- **Células de texto (Markdown)**: normalização Unicode NFKC, minúsculas, separação controlada da pontuação e compactação de espaços; **acentos são preservados**.
- **Células de código**: a fonte é guardada intacta e ganha uma visão lexical à parte (tokens), sem alterar a indentação.
- **Stopwords não são removidas** na configuração principal: em português isso apagaria termos funcionais (preposições, conjunções) que mudam o sentido de consultas como “*diferença entre while e for*”.
- Saídas de execução das células são descartadas, para não introduzir texto derivado de execução.

5.2 Segmentação (chunking)

A segmentação respeita a hierarquia do livro — notebook, título e seção — e **nunca mistura** célula de texto com célula de código no mesmo chunk. A configuração principal usa **256 tokens** com **sobreposição de 32**, o que produz cerca de **330 chunks**. As configurações 128/16 e 512/64 entram apenas no experimento de sensibilidade ao tamanho do bloco.

5.3 Representações computacionais

- **Saco de palavras / TF-IDF:** cada chunk é um vetor esparsos de pesos TF-IDF sobre o vocabulário V .
- **Índice invertido (Config. 2):** um mapa termo $\rightarrow$ lista de *postings* (id do chunk, $f_{t,c}$), com o **vocabulário mantido em um vetor ordenado** para permitir **busca binária** por termo.

Riscos conhecidos do corpus. A tradução ainda carrega termos em inglês do Python/Jupyter (ex.: *loop*, *string*), o que afeta consultas com e sem estrangeirismos; um único livro introdutório não representa todo o universo dos materiais educacionais abertos — os resultados valem como **estudo de caso**.

6. Algoritmos e pseudocódigo

Os algoritmos obrigatórios do enunciado (§5.2) são cobertos assim: **busca linear** na pontuação de todos os chunks; **ordenação** por Insertion Sort e Merge Sort implementados pela equipe; **busca binária** no vocabulário do índice invertido; **divisão e conquista** pelo próprio Merge Sort; e **comparação com biblioteca** pelo TF-IDF do scikit-learn, usado apenas como referência externa (Config. 4).

6.1 Busca linear com pontuação (comum a C1 e C3)

```
função PontuarLinear(q, C, idf):
    candidatos ← lista vazia
    para cada chunk c em C faça           # N iterações
        s ← 0
        para cada termo t em q faça       # m iterações
            se t ∈ c.termos então         # teste O(1) via hash
                s ← s + tf(t, c) · idf[t]
            se s > 0 então candidatos.anexar( (s, c.id, c) )
    retorne candidatos
```

6.2 Insertion Sort — Config. 1

```
função InsertionSort(A, ≤):
    para i ← 1 até |A|-1 faça
        chave ← A[i]; j ← i - 1
        enquanto j ≥ 0 e A[j] > chave faça # desloca maiores
            A[j+1] ← A[j]; j ← j - 1
        A[j+1] ← chave
    retorne A
```

6.3 Merge Sort — Config. 3 e estratégia de divisão e conquista

```
função MergeSort(A, ≤):
    se |A| ≤ 1 então retorne A           # caso base
    meio ← ⌊|A| / 2⌋
    E ← MergeSort(A[0 .. meio-1], ≤)    # T(N/2)
    D ← MergeSort(A[meio .. |A|-1], ≤)  # T(N/2)
    retorne Merge(E, D, ≤)              # Θ(N)

função Merge(E, D, ≤):
    O ← lista vazia; i ← 0; j ← 0
    enquanto i < |E| e j < |D| faça
        se E[i] ≤ D[j] então O.anexar(E[i]); i ← i+1 # ≤ estável: empate → E
        senão O.anexar(D[j]); j ← j+1
    O.anexar(restante de E a partir de i)
    O.anexar(restante de D a partir de j)
    retorne O
```

6.4 Índice invertido e busca binária — Config. 2

```
função BuscaBinaria(vocab, t):           # vocab é vetor ORDENADO de termos
    lo ← 0; hi ← |vocab| - 1
    enquanto lo ≤ hi faça
        mid ← ⌊(lo + hi) / 2⌋
        se vocab[mid] = t então retorne mid
        senão se vocab[mid] < t então lo ← mid + 1
        senão hi ← mid - 1
    retorne NÃO_ENCONTRADO

função ConsultarIndice(q, vocab, postings, idf):
    acc ← mapa id → score (vazio)
    para cada termo t em q faça           # m termos
        p ← BuscaBinaria(vocab, t)       # Θ(Log V)
        se p ≠ NÃO_ENCONTRADO então
            para cada (id, f) em postings[t] faça # só os candidatos!
                acc[id] ← acc[id] + (1 + ln f) · idf[t]
    candidatos ← [ (s, id, chunk(id)) para (id, s) em acc ]
    retorne MergeSort(candidatos, ≤)     # ordena só os candidatos
```

6.5 Seleção dos k melhores (comum a todas)

```
função TopK(candidatos_ordenados, k):  
    retorne os primeiros min(k, |candidatos_ordenados|) elementos
```

Como a lista já está totalmente ordenada por $\preceq$, os primeiros $\min(k, n)$ elementos são, por definição, os k mais relevantes (corolário provado na Seção 7).

7. Justificativa de corretude

Provamos a corretude do **Merge Sort** (Config. 3 e núcleo da Config. 2), por indução forte no tamanho da entrada, apoiada em um invariante de laço para o procedimento `Merge`. O texto completo está em `docs/CORRETUDE.md`.

Teorema. Dada uma lista finita A de chunks com identificadores únicos e a ordem total $\preceq$ (score decrescente; empate por id crescente), `MergeSort(A,  $\preceq$ )` **termina** e devolve uma **permutação de A ordenada** por $\preceq$.

Hipóteses

- A é finita; cada elemento tem score real finito e id único.
- $\preceq$ é uma **ordem total**: reflexiva, antissimétrica (garantida pelo desempate por id único) e transitiva.
- A comparação de dois elementos custa $\Theta(1)$.

Lema (corretude do `Merge`) — invariante de laço

Invariante. No início de cada iteração do laço **enquanto** de `Merge(E, D)`, com E e D já ordenados por $\preceq$: (i) a saída O está ordenada por $\preceq$; (ii) O contém exatamente os elementos já consumidos de E (índices $< i$) e de D (índices $< j$); e (iii) todo elemento de O é $\preceq$ a qualquer elemento ainda não consumido de E ou D .

- Inicialização.** Antes da primeira iteração, $O = \langle \rangle$ e $i = j = 0$. (i)–(iii) valem vacuamente.
- Manutenção.** Suponha o invariante no início de uma iteração. O algoritmo escolhe o menor entre $E[i]$ e $D[j]$ (empate resolvido a favor de E). Como E e D estão ordenados, o elemento escolhido x é $\preceq$ a todos os demais não consumidos de E e de D ; logo pode ser anexado ao fim de O preservando a ordenação (i) e a propriedade (iii). O contador correspondente avança, mantendo (ii). Portanto o invariante vale no início da próxima iteração.
- Término.** Cada iteração consome exatamente um elemento, então o laço executa no máximo $|E| + |D|$ vezes e termina quando E ou D se esgota. Os elementos restantes da lista não esgotada já são, por (iii), $\preceq$ a nada em O e estão em ordem; anexá-los mantém O ordenado. Ao final, O contém todos os $|E| + |D|$ elementos, ordenados — uma permutação ordenada de $E \cup D$.

Indução no tamanho de A

- Base ($|A| \leq 1$).** A lista é devolvida como está: trivialmente ordenada e permutação de si mesma. Termina sem recursão.
- Hipótese de indução.** `MergeSort` é correto para toda entrada de tamanho $< n$.
- Passo ($|A| = n > 1$).** A é dividida em E ($\lfloor n/2 \rfloor$) e D ($n - \lfloor n/2 \rfloor$), ambas de tamanho $< n$. Por hipótese de indução, as chamadas recursivas devolvem permutações ordenadas de E e de D . Pelo Lema, `Merge(E, D)` devolve uma permutação ordenada de $E \cup D$. Como a concatenação de E e D é a própria A a menos de reordenação, o resultado é uma permutação ordenada de A . ■

Término da recursão

Os tamanhos das subchamadas decrescem estritamente ($\lfloor n/2 \rfloor < n$ e $n - \lfloor n/2 \rfloor < n$ para $n > 1$) e sempre atingem o caso base; logo a recursão é finita.

Estabilidade e desempate

O `Merge` anexa de E em caso de empate, tornando a ordenação **estável**. Como os id são únicos e entram na própria relação $\preceq$, não há empates verdadeiros entre chunks distintos: a saída é **determinística**, independentemente da ordem de entrada e da configuração (C_1 , C_2 ou C_3). Isso é verificado por testes que ordenam a mesma entrada embaralhada e comparam a saída.

Corolário (top- k). Se A está ordenada por $\preceq$, então seus primeiros $\min(k, |A|)$ elementos são exatamente os k maiores segundo $\preceq$. Logo `TopK` aplicado à saída do `MergeSort` devolve os k chunks mais relevantes, cumprindo a pós-condição da Seção 4.3.

Limites do argumento e casos em que não se aplica

- A prova garante **ordenação correta e término**, não que a função TF-IDF corresponda à intenção de quem consultou: relevância lexical é um *proxy* imperfeito (ver Seções 14 e 15).
- Se algum score for indefinido (NaN) ou o comparador não for transitivo, $\preceq$ deixa de ser ordem total e o teorema não se aplica.
- Se os id não forem únicos, o desempate falha e a saída pode deixar de ser determinística.
- A prova não fala de precisão numérica de ponto flutuante; scores muito próximos podem, na prática, ser tratados como empate e então decididos pelo `source_order` — comportamento coberto pelos testes de desempate.

Validação prática da ordenação

Como complemento à prova, a relação de ordem $\preceq$ usada em todas as configurações é validada diretamente em `tests/unit/test_sort_key.py`, sobre a função `sort_key` de `src/paa_context/sort_key.py`, que estabelece maior `score` primeiro e, em empate, menor `source_order`. Três testes cobrem os pilares do argumento:

- `test_maior_score_vem_primeiro`: prioridade do maior `score`;
- `test_empate_usa_menor_source_order`: critério de desempate por `source_order`;
- `test_ordem_transitiva`: transitividade da relação (condição para $\preceq$ ser ordem total).

Os três passam isoladamente e a suíte completa do projeto é executada e aprovada (87 testes na versão de 23/09/2026 — atualizar o número na versão congelada). Esta análise de corretude, modelo RAM, complexidade e recorrências foi conduzida por Nadianne Galvão (docs/CORRETUDE.md e o documento de corretude/RAM/complexidade/recorrências).

8. Análise no modelo RAM

8.1 Parâmetros

Símbolo	Significado
N	número de chunks (~330 na configuração principal)
V	tamanho do vocabulário (termos distintos)
L	total de tokens do corpus
m	número de termos da consulta
k	número de resultados retornados
C_q	número de chunks candidatos (com score > 0) para a consulta q
P	tamanho total das listas de postings percorridas = $\sum_{t \in q} df(t)$
r	número de repetições experimentais

8.2 Operações elementares e custo por algoritmo

Contam-se como operações elementares: comparação de chaves, acesso a vetor/lista, atribuição, incremento, chamada recursiva, consulta a tabela de dispersão e atualização de estrutura. Para dois algoritmos, o detalhamento:

Insertion Sort (sobre C_q candidatos)

O laço externo executa $C_q - 1$ vezes; o laço interno desloca, no pior caso, todos os elementos já ordenados. O número de comparações é $\sum_{i=1}^{C_q-1} i = C_q(C_q - 1)/2$ no pior caso, com igual ordem de grandeza de acessos e atribuições. Espaço auxiliar $\Theta(1)$ (ordena no lugar).

Merge Sort (sobre C_q candidatos)

Cada nível da recursão percorre todos os elementos uma vez no Merge ($\Theta(C_q)$ comparações e cópias por nível) e há $\lceil \log_2 C_q \rceil$ níveis, dando $\Theta(C_q \log C_q)$ comparações. O Merge usa listas auxiliares, logo o espaço é $\Theta(C_q)$.

Custo do pipeline por configuração

Etapas	C1 (linear + Insertion)	C2 (índice + binária + Merge)	C3 (linear + Merge)
Pontuação / seleção de candidatos	$\Theta(N \cdot m)$	$\Theta(m \cdot \log V + P)$	$\Theta(N \cdot m)$
Ordenação dos candidatos	$\Theta(C_q^2)$	$\Theta(C_q \log C_q)$	$\Theta(C_q \log C_q)$
Seleção top-k	$\Theta(k)$	$\Theta(k)$	$\Theta(k)$
Espaço adicional (consulta)	$\Theta(C_q)$	$\Theta(V + P_{\text{total}})$ do índice + $\Theta(C_q)$	$\Theta(C_q)$

Fatores fora do modelo RAM. O modelo ignora hierarquia de cache, custo de E/S ao ler os notebooks, alocação de memória, comportamento do coletor de lixo do Python e a diferença entre código interpretado (C1–C3) e rotinas vetorizadas em C/NumPy (Config. 4 do scikit-learn). Por isso a Seção 12 mede tempo de parede além de contar operações.

9. Melhor, pior e caso médio

Algoritmo	Melhor caso	Pior caso	Caso médio	Espaço
Busca/pontuação linear	$\Theta(N \cdot m)$	$\Theta(N \cdot m)$	$\Theta(N \cdot m)$	$\Theta(C_q)$
Insertion Sort	$\Theta(n)$ — já ordenado	$\Theta(n^2)$ — invertido	$\Theta(n^2)$	$\Theta(1)$
Merge Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$
Busca binária (por termo)	$\Theta(1)$	$\Theta(\log V)$	$\Theta(\log V)$	$\Theta(1)$
Consulta via índice (C2)	$\Theta(m \log V + C_q \log C_q)$	$\Theta(m \log V + N \log N)$	$\Theta(m \log V + P + C_q \log C_q)$	$\Theta(V + P_{\text{total}})$

Onde $n = C_q$ é o número de candidatos a ordenar. Observações sobre os parâmetros relevantes:

- **Insertion Sort** só é competitivo quando C_q é pequeno ou a lista já está quase ordenada; seu melhor caso linear ocorre quando nenhum deslocamento é necessário.
- **Merge Sort** tem custo $\Theta(n \log n)$ **garantido** nos três casos, independentemente da distribuição — o que o torna a escolha segura para a comparação.

- **Consulta via índice (C2)** tem melhor caso quando os termos são **raros** (P e C_q pequenos) e degenera ao custo de C_3 no pior caso, quando os termos são tão frequentes que $C_q \approx N$ (por exemplo, uma stopword muito comum). É por isso que a hipótese de esparsidade (Seção 4.3) é o que separa C_2 de C_3 na prática.

10. Recorrências

10.1 Merge Sort

$$T(N) = 2 \cdot T(N/2) + \Theta(N), \quad T(1) = \Theta(1)$$

Origem dos termos na implementação: o fator **2** vem das duas chamadas recursivas (metades E e D); cada $T(N/2)$ é o custo de ordenar uma metade; o termo $\Theta(N)$ é o custo do **Merge**, que percorre linearmente as duas metades uma única vez. Pela **árvore de recorrência** (Figura 2), cada um dos $\lceil \log_2 N \rceil + 1$ níveis realiza trabalho total $\Theta(N)$, somando $\Theta(N \log N)$. O **Teorema Mestre** confirma: com $a = 2$, $b = 2$, tem-se $N^{\log_b a} = N^1$, e como $f(N) = \Theta(N) = \Theta(N^1)$, aplica-se o caso 2, dando $T(N) = \Theta(N \log N)$.

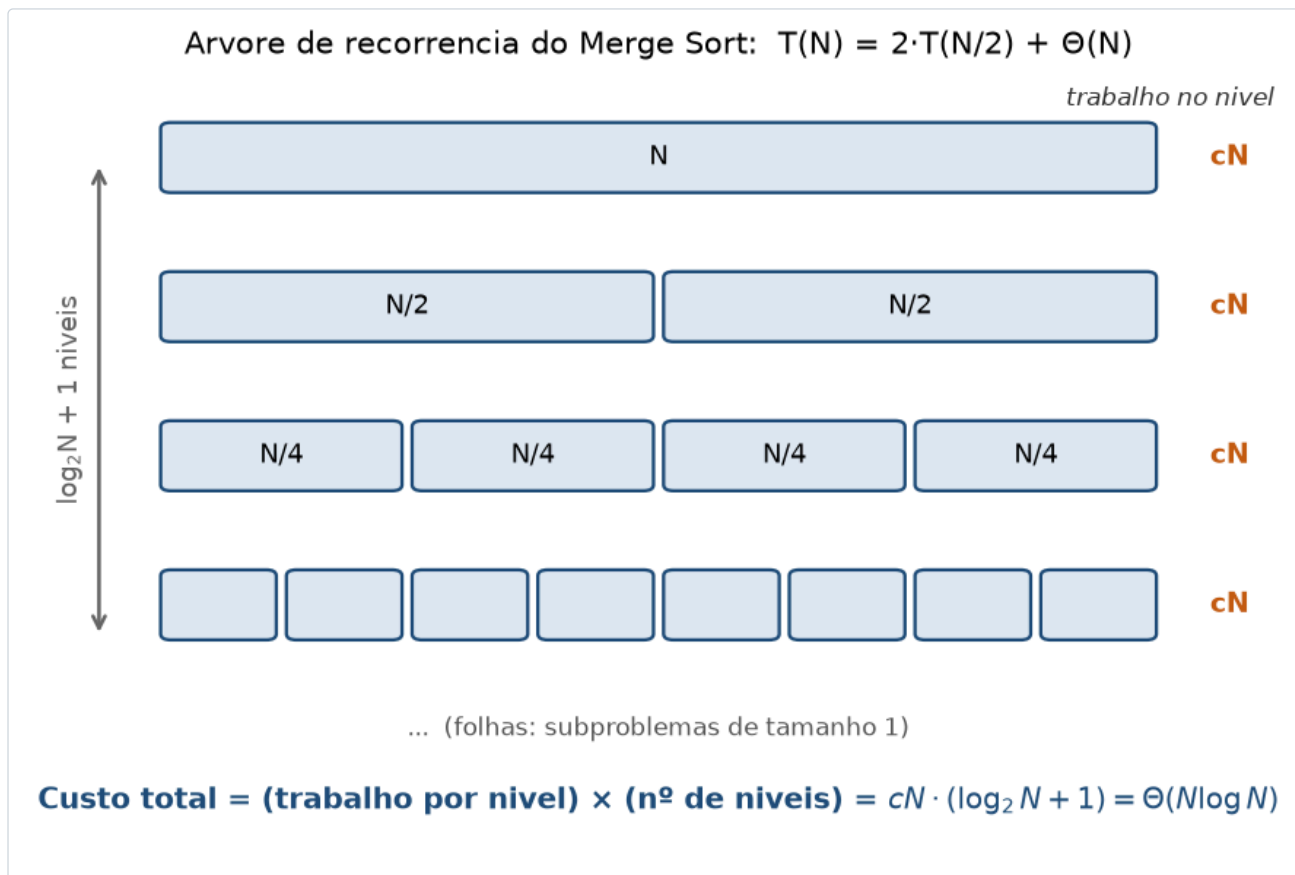


Figura 2 — Árvore de recorrência de $T(N) = 2 \cdot T(N/2) + \Theta(N)$. O trabalho por nível é constante em $\Theta(N)$ e há $\log_2 N + 1$ níveis, resultando em $\Theta(N \log N)$. Diagrama teórico.

10.2 Busca binária

$$T(V) = T(V/2) + \Theta(1), \quad T(1) = \Theta(1)$$

Origem dos termos: há uma única chamada recursiva sobre metade do vocabulário (a outra metade é descartada a cada passo), e o termo $\Theta(1)$ corresponde ao cálculo do meio e à comparação com `vocab[mid]`. Pelo Teorema Mestre, $a = 1$, $b = 2$, $N^{\log_b a} = N^0 = 1 = f(N)$: caso 2 novamente, dando $T(V) = \Theta(\log V)$.

11. Metodologia experimental

11.1 Configurações comparadas

Config.	Descrição	Papel
C1 — baseline	Varredura linear + Insertion Sort (implementado pela equipe)	Baseline sem estrutura
C2 — indexada	Índice invertido + busca binária no vocabulário + Merge Sort dos candidatos	Busca indexada/otimizada
C3 — ordenada	Varredura linear + Merge Sort (implementado pela equipe)	Divisão e conquista na ordenação
C4 — referência	TF-IDF do scikit-learn (biblioteca)	Somente baseline externo; não substitui nenhuma exigência

As três configurações obrigatórias usam **os mesmos dados e a mesma função de pontuação**; diferem apenas na estrutura de busca e na ordenação. Consequência importante para a validação: **C1, C2 e C3 devem produzir listas ranqueadas idênticas** (mesma ordem total $\leq$), diferindo apenas em custo — um teste de consistência forte. A Config. 4 pode divergir por usar normalização L2 e TF sublinear próprias, e por isso é analisada em separado.

11.2 Cargas, repetições e execuções

O plano cobre **3 configurações × 3 tamanhos de corpus × 5 repetições = 45 execuções mensuráveis** (bem acima do mínimo de 12 exigido, e com folga sobre as “pelo menos duas repetições por cenário”). Os três tamanhos são obtidos por **subamostragem determinística** (semente fixa) de aproximadamente 25%, 50% e 100% dos chunks; adicionalmente, o experimento de sensibilidade varia o chunking em 128/16, 256/32 e 512/64. Cada consulta é executada para **k = 5 e k = 10**. As duas primeiras execuções de cada cenário são descartadas como aquecimento e a mediana das repetições é reportada com a dispersão.

11.3 Métricas registradas

- Tempo de pré-processamento/ingestão; tempo de ordenação ou de construção do índice; tempo de consulta; tempo total.
- Uso aproximado de memória (via `tracemalloc` e RSS do processo).
- **Número de comparações**, por contador instrumentado nos algoritmos de ordenação e busca.
- Tamanho do corpus, número de chunks e k.
- **Precision@k** contra os julgamentos de relevância (`data/qrels.csv`), quando há referência.
- Taxa de falhas, resultados vazios ou itens irrelevantes; custo e limitações de cada abordagem.

11.4 Reprodutibilidade

Todo o pipeline roda com `python scripts/reproduzir_tudo.py` (download → normalização → chunking → experimentos → figuras). Os scripts são **idempotentes**. Cada execução registra em `experimentos/logs`: hostname, processador, memória, sistema operacional, versão do Python, versões das bibliotecas, commit e a linha de comando completa — atendendo à exigência de registrar o ambiente. As consultas e os qrels foram fixados **antes** de observar qualquer resultado, para evitar viés de seleção.

11.5 Julgamentos de relevância

As 30 consultas de `data/queries.csv` recebem julgamentos em escala 0, 1 e 2 por **dois avaliadores independentes**, com a adjudicação dos desacordos registrada em `data/qrels.csv`. Isso dá a referência de relevância necessária para o Precision@k e permite reportar a concordância entre avaliadores.

12. Resultados e gráficos

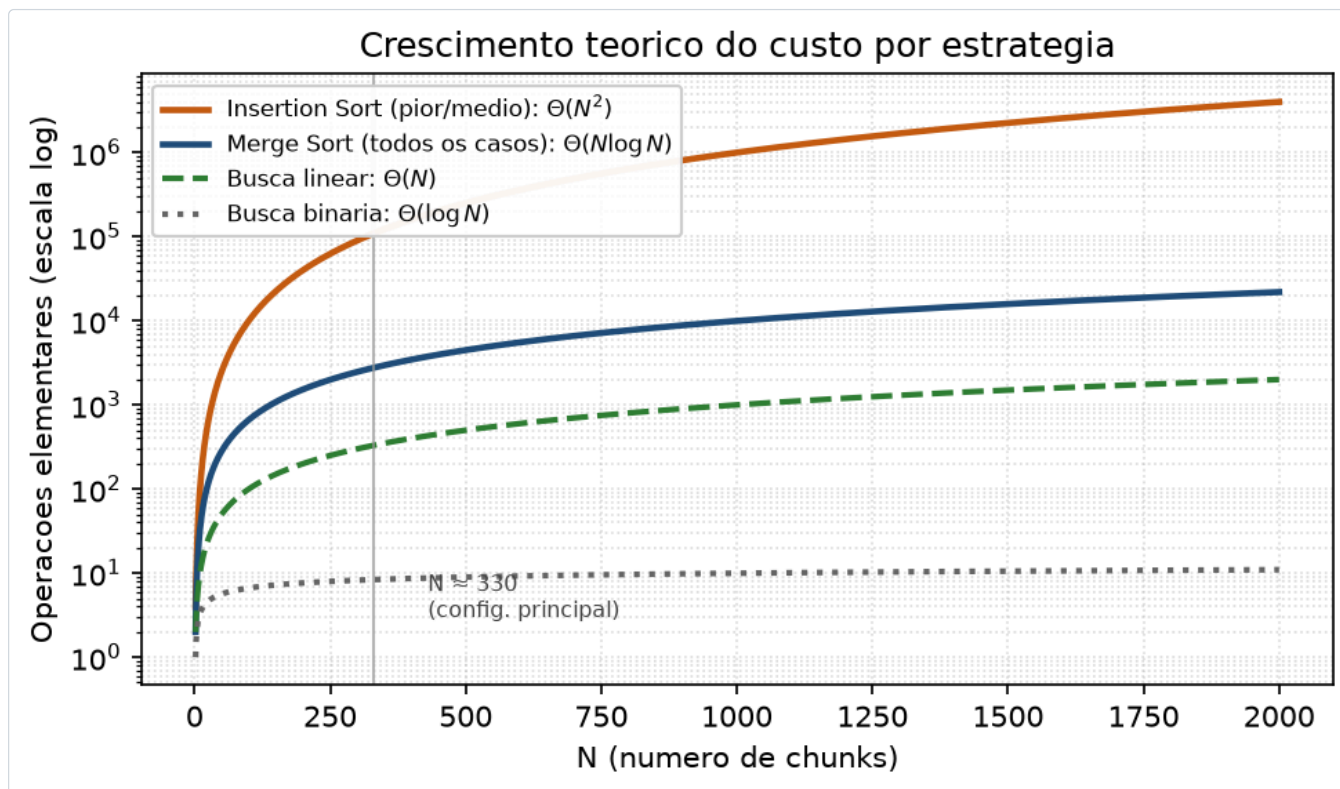


Figura 1 — Crescimento teórico do número de operações elementares em função de N (escala logarítmica no eixo y). A separação entre $\Theta(N^2)$ (Insertion), $\Theta(N \log N)$ (Merge) e $\Theta(\log N)$ (busca binária) motiva as três configurações. Curvas teóricas, não medições.

Tabelas empíricas — a preencher após a execução. A estrutura de colunas abaixo é a definitiva; os valores serão inseridos a partir de experimentos/processados após rodar `scripts/rodar_experimentos.py`. A previsão qualitativa (não medida) que os experimentos vão testar está indicada em itálico abaixo de cada tabela.

12.1 Tempo por etapa e configuração (mediana de r repetições)

Config.	N (chunks)	Ingestão (ms)	Ordenação/índice (ms)	Consulta (ms)	Total (ms)	Comparações
C1	~330	—	—	—	—	—
C2	~330	—	—	—	—	—
C3	~330	—	—	—	—	—
C4 (ref.)	~330	—	—	—	—	n/a

Tabela 1 — Tempos e contagem de comparações. Valores a preencher (mediana; reportar também dispersão).

Previsão a testar: o tempo de ordenação de $C1$ cresce como C_q^2 , superando $C2/C3$ conforme o número de candidatos aumenta; a consulta de $C2$ é a menor por tocar apenas os candidatos; a contagem de comparações de $C2$ e $C3$ fica próxima de $C_q \cdot \log_2 C_q$ e a de $C1$ próxima de $C_q^2/2$ no pior caso.

12.2 Qualidade da recuperação e memória

Config.	Precision@5	Precision@10	Taxa de vazios	Memória pico (MB)
C1	—	—	—	—
C2	—	—	—	—
C3	—	—	—	—
C4 (ref.)	—	—	—	—

Tabela 2 — Qualidade e memória. Valores a preencher.

Previsão a testar: $C1$, $C2$ e $C3$ têm **Precision@ k idêntica** (mesma função de pontuação e mesma ordem total), confirmando que a diferença entre elas é de custo, não de qualidade; $C2$ consome mais memória por manter o índice invertido; $C4$ pode divergir na Precision@ k por usar normalização TF-IDF diferente.

12.3 Escalabilidade medida (a preencher)

N (chunks)	C1 total (ms)	C2 total (ms)	C3 total (ms)
~82 (25%)	—	—	—
~165 (50%)	—	—	—
~330 (100%)	—	—	—

Tabela 3 — Tempo total por tamanho de corpus. O gráfico empírico correspondente será gerado por `scripts/gerar_figuras.py` e substituirá este marcador.

Previsão a testar: a curva de C_1 acompanha o crescimento super-linear do Insertion Sort sobre os candidatos, enquanto C_2 e C_3 crescem de forma quase linear-logarítmica; a vantagem de C_2 se amplia com N por evitar tocar chunks sem os termos da consulta.

13. Discussão de escalabilidade

Para o corpus atual (~330 chunks) as três configurações rodam em milissegundos e as diferenças de tempo são pequenas em valor absoluto — mas a **tendência** é o que interessa. À medida que o acervo cresce (imagine dezenas de livros abertos, com N na casa dos milhares ou dezenas de milhares de chunks):

- **C1** torna-se inviável: o Insertion Sort sobre C_q candidatos cresce como C_q^2 , e a varredura linear toca todos os N chunks a cada consulta.
- **C2** desacopla o custo de consulta do tamanho total do corpus: passa a depender da **seletividade dos termos** (P e C_q) e de $\log V$, não de N . É a única das três que escala para acervos grandes — ao custo de memória extra para o índice invertido ($\Theta(V + P_{\text{total}})$).
- **C3** mantém a ordenação em $\Theta(N \log N)$, mas ainda paga a varredura linear $\Theta(N \cdot m)$ por consulta.

Esse é exatamente o trade-off central de sistemas de recuperação: gastar memória e tempo de indexação uma vez para tornar cada consulta sublinear. Bibliotecas como FAISS levam essa ideia ao caso vetorial denso, inclusive para coleções que não cabem em RAM; aqui a demonstramos no caso lexical, com estruturas clássicas.

14. Relação com IA generativa e RAG

O contexto recuperado pelo protótipo é precisamente a etapa de *retrieval* de um sistema RAG (*Retrieval-Augmented Generation*). Discutimos os sete pontos pedidos:

1. **Incorporação ao prompt.** Os k chunks retornados seriam concatenados em um bloco de contexto — “Contexto: <chunks>\n\nPergunta: <q>” — cada trecho acompanhado da sua origem (notebook, seção) para permitir citação.
2. **Impacto de k , tamanho de chunk e ordenação no custo de contexto.** Mais chunks e chunks maiores gastam mais tokens da janela do LLM, aumentando latência e custo de geração. A ordem também importa: modelos tendem a subaproveitar informação no meio do contexto (*lost in the middle*), então colocar os trechos mais relevantes primeiro (o que nossa ordem $\Leftarrow$ garante) é vantajoso.
3. **Lexical vs. semântico.** O TF-IDF captura **sobreposição de termos**, não sinônimos nem paráfrases. No nosso corpus em português com termos técnicos em inglês, uma consulta por “*laço*” pode não casar com um trecho que diz “*loop*” — caso em que a busca semântica com embeddings recuperaria o trecho e a lexical não.
4. **Riscos de recuperação ruim.** Recuperação **incompleta** (o trecho certo não está entre os k), **enviesada** (um único livro cobre só uma visão), **desatualizada** (commit fixo — desatualização controlada, mas real) ou **irrelevante** comprometem diretamente a resposta final.
5. **Afirmações não sustentadas.** Mesmo com bom contexto, o LLM pode “alucinar” além do que foi recuperado. Mitiga-se com instrução explícita de responder apenas com base no contexto e de citar a origem de cada afirmação.
6. **Rastreabilidade.** Como cada chunk carrega a proveniência (notebook, seção), a resposta pode citar a fonte exata, permitindo ao usuário verificar — melhoria direta de confiabilidade sobre um LLM sem recuperação.
7. **Trade-offs.** Há tensão entre precisão, tempo de resposta, uso de memória e custo de geração: aumentar k eleva o *recall* potencial, mas também o custo em tokens e o ruído; indexar (C2) acelera a consulta ao preço de memória.

15. Limitações e ameaças à validade

- **Validade de construção.** A Precision@k depende de julgamentos de relevância subjetivos; mitigamos com dois avaliadores e adjudicação, mas a referência não é perfeita.
- **Validade interna.** Medições de tempo em Python sofrem com coletor de lixo, cache e aquecimento; por isso reportamos medianas de repetições, descartamos as duas primeiras execuções e contamos operações elementares como métrica independente do hardware.
- **Validade externa.** Um único livro introdutório em português, do domínio de programação, não generaliza para todo o universo dos materiais educacionais abertos; os resultados valem como estudo de caso.
- **Viés do corpus.** A tradução mistura português e termos técnicos em inglês, o que penaliza a busca lexical em consultas que usam apenas uma das formas.
- **Validade de conclusão.** Todas as medidas vêm de um único ambiente de hardware; diferenças pequenas de tempo devem ser lidas com cautela e sempre junto da análise assintótica.

16. Declaração de uso de IA generativa

A declaração detalhada é mantida **durante** o trabalho em `docs/USO_DE_IA.md`, com ferramenta, modelo, finalidade, até cinco prompts reais, sugestões aproveitadas, sugestões corrigidas ou rejeitadas, erros identificados e a verificação aplicada. Resumo do uso até esta versão:

Ferramenta / modelo	Finalidade	Verificação exigida
Assistente de IA generativa (a registrar com o modelo exibido na interface)	Apoio à estruturação e redação deste relatório e à formatação da prova de corretude e dos pseudocódigos.	Toda saída de IA será conferida pela equipe quanto a corretude, complexidade, casos de borda, estabilidade de ordenação e desempate, e coerência entre a análise formal e o experimento, conforme §9 do enunciado. Código, prova ou análise não verificados não entram no trabalho.

Compromisso de uso crítico. A prova de corretude da Seção 7 será validada por testes que exercitam estabilidade e desempate; os pseudocódigos, por implementação e testes unitários; e as previsões da Seção 12, pela execução real. O arquivo `docs/USO_DE_IA.md` registra cada uso no dia em que ocorre, com o responsável.

17. Contribuição individual

O enunciado pede contribuição **com evidência**, não cargo. A tabela final, com commits, arquivos, análises e execuções que comprovam cada frente, é mantida em `docs/CONTRIBUICOES.md` e será consolidada na entrega. A distribuição de frentes proposta consta na Seção 1. Todos os integrantes participam da revisão coletiva do código, da prova e dos resultados, e da apresentação e do vídeo.

Integrante	Frente	Evidências (a consolidar)
João Cosme Sena Sá	Índice invertido / busca binária (C2); dados de teste, pacote e protocolo	<code>src/paa_context/indice.py</code> , <code>busca_binaria.py</code> , <code>modelos.py</code> , <code>fixtures</code>
Eduardo Henrique do Lago Silva	Experimentos / figuras / baseline sklearn (C4)	<code>rodar_experimentos.py</code> , <code>gerar_figuras.py</code> , análise de C4
Helena Carvalho Leal	Consultas / qrels / métricas / IA / limitações	<code>data/queries.csv</code> , <code>data/qrels.csv</code> , <code>metricas.py</code>
Hernandison da Silva Bispo	Ingestão / normalização / fragmentação / relatório	<code>preprocessing.py</code> , <code>fragmentacao.py</code> , <code>scripts/relatorio/</code>
Nadianne Maria dos Santos Galvão	Corretude / RAM / complexidade / recorrências	<code>docs/CORRETUDE.md</code> , <code>test_sort_key.py</code> , Seções 7–10
Rafael Takeguma Goto	Busca linear / Insertion / Merge / pontuação (C1 e 3)	<code>linear_search.py</code> , <code>insertion_sort.py</code> , <code>merge_sort.py</code> , <code>relevance.py</code>

18. Vídeo da atividade

URL: a preencher até 22/09/2026. O vídeo terá até 10 minutos, com participação de todos os integrantes e identificação clara de cada contribuição. A mesma URL constará, obrigatoriamente, no `README.md` (seção “Vídeo da atividade”), na capa deste relatório, no arquivo `VIDEO.md` (com data de gravação e participantes) e na área da atividade no Google Classroom quando solicitado. O vídeo apresentará: identificação da equipe, tema e corpus; o problema de recuperação; os algoritmos; uma demonstração breve do protótipo; a justificativa de corretude; a análise assintótica e as recorrências; os resultados e o gráfico principal; a relação com RAG; e as limitações e decisões de projeto.

19. Referências

- CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms*. 4. ed. Cambridge: MIT Press, 2022.
- KLEINBERG, J.; TARDOS, É. *Algorithm Design*. Boston: Pearson, 2006.
- SKIENA, S. S. *The Algorithm Design Manual*. 3. ed. Cham: Springer, 2020.
- SEDGEWICK, R.; WAYNE, K. *Algorithms*. 4. ed. Boston: Addison-Wesley, 2011.
- MANNING, C. D.; RAGHAVAN, P.; SCHÜTZE, H. *Introduction to Information Retrieval*. Cambridge University Press, 2008. (TF-IDF e índice invertido.)
- LEWIS, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*, 2020. arXiv:2005.11401.
- ZHOU, Z. et al. A Survey on Efficient Inference for Large Language Models. arXiv:2404.14294, 2024.
- GAN, A. et al. Retrieval Augmented Generation Evaluation in the Era of Large Language Models: A Comprehensive Survey. arXiv:2504.14891, 2025.
- FAISS. *Faiss documentation*. Disponível em: <https://faiss.ai/>.
- SENTENCE TRANSFORMERS. Repositório oficial. Disponível em: <https://github.com/huggingface/sentence-transformers>.
- PEDREGOSA, F. et al. Scikit-learn: Machine Learning in Python. *JMLR*, v. 12, 2011. (Baseline TF-IDF — Config. 4.)
- DOWNEY, A. B. *Think Python*. 3. ed. O'Reilly, 2024. Tradução livre *Pense Python* por CARLSON, R. C. Disponível em: [https://rodrigocarlson.github.io/PensePython3ed/\(corpus\)](https://rodrigocarlson.github.io/PensePython3ed/(corpus)).